# 📚 Penjelasan Lengkap Sistem Chatbot UPJ

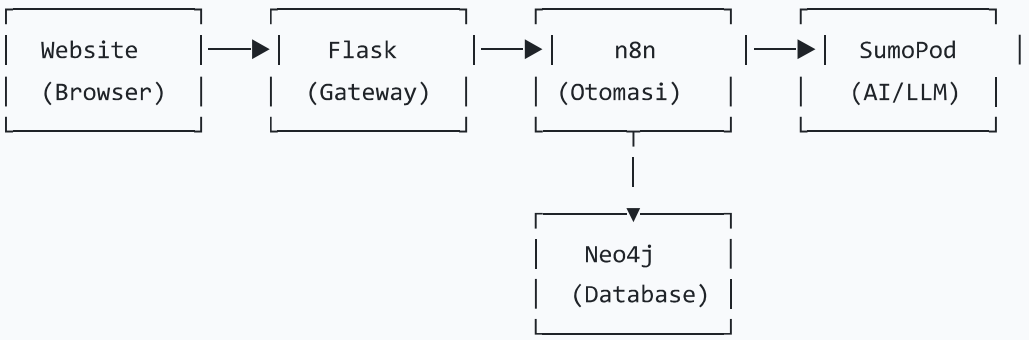
> Panduan untuk pemula — tidak perlu latar belakang teknis

## 🗺️ Gambaran Besar: Apa yang Dibangun?

Kita membangun sebuah **chatbot pintar untuk informasi kampus UPJ** yang bisa menjawab pertanyaan seperti:

- "Berapa biaya kuliah Informatika?"
- "Apa saja beasiswa di UPJ?"
- "Siapa dosen di prodi Sistem Informasi?"

Sistem ini terdiri dari **4 komponen utama** yang bekerja sama:




## 🔄 Alur Percakapan — Step by Step

Bayangkan pengguna mengetik: **"Berapa biaya kuliah Informatika?"**

```
[1] Pengguna ketik pertanyaan di website
        ↓
[2] Browser kirim ke Flask (localhost:5000)
        ↓
[3] Flask teruskan ke n8n (localhost:5678)
        ↓
[4] n8n: Build Cypher Query
    → Deteksi kata "biaya" → buat query database
        ↓
[5] n8n: Query Neo4j (localhost:7474)
    → Dapat data biaya kuliah dari database
        ↓
[6] n8n: Format Prompt
    → Gabungkan data + pertanyaan → buat instruksi untuk AI
        ↓
[7] n8n: Kirim ke SumoPod AI
    → AI baca data + pertanyaan → buat jawaban natural
        ↓
[8] n8n: Kirim jawaban balik ke Flask
        ↓
[9] Flask teruskan ke Browser
        ↓
[10] Pengguna lihat jawaban di website
```

## 📁 File-File dalam Proyek

```
d:\techsolusi\RAG\
│
├── 🕷️  scrape_pmb_upj.py          ← Pengumpul data dari website PMB
├── 🏗️  build_neo4j_graph.py        ← Penyimpan data ke database Neo4j
├── 🌐  app.py                      ← Server web (Flask gateway)
├── 📋  n8n_chatbot_workflow.json   ← Resep alur kerja n8n
│
├── templates/
│   └── 🎨  index.html              ← Tampilan website chatbot
│
└── 📦  scraped_*.json              ← Hasil scraping (data mentah)
```

## 📄 Penjelasan Setiap File

### 1. 🕷️ `scrape_pmb_upj.py` — Pengumpul Data

**Analogi:** Seperti asisten yang disuruh baca semua halaman website PMB UPJ dan mencatat informasinya ke buku catatan.

**Yang dilakukan:**

- Mengunjungi halaman-halaman di `pmb.upj.ac.id` secara otomatis
- Membaca dan menyimpan teks dari setiap halaman
- Mengambil informasi: biaya kuliah, beasiswa, syarat masuk, jalur pendaftaran, FAQ

**Hasil:** File-file `.json` berisi data terstruktur, contoh:

```
json
{
  "biaya_kuliah": [
    {
      "prodi": "Informatika",
      "spp": "Rp 3.500.000/semester",
      "total_semester_1": "Rp 9.450.000"
    }
  ],
  "beasiswa": [
    {
      "nama": "Beasiswa Jaya 100%",
      "syarat": "Nilai rapor rata-rata 85 ke atas"
    }
  ]
}
```

### 2. 🧬 `build_neo4j_graph.py` — Pembangun Database

**Analogi:** Seperti pustakawan yang mengambil semua buku catatan (JSON) dan menyusunnya dalam perpustakaan pintar (Neo4j) agar mudah dicari dan dihubungkan.

**Yang dilakukan:**

- Membaca semua file JSON hasil scraping

- Membuat **hubungan antar data** — misalnya: "Prodi Informatika **memiliki** biaya kuliah X" dan "Prodi Informatika **diajarkan oleh** dosen Y"
- Menyimpan semua ke database Neo4j

**Data yang disimpan:**

| Data | Jumlah |
|---|---|
| Program Studi | 11 prodi |
| Dosen | 101 orang |
| Mata Kuliah | 924 mata kuliah |
| Halaman Web | 473 halaman |
| Beasiswa | 9 jenis |
| FAQ | 6 pertanyaan umum |

**Cara jalankan:**

```
powershell
python build_neo4j_graph.py --password password123
```

### 3. 🌐 `app.py` — Server Web (Pintu Gerbang)

**Analogi:** Seperti resepsionis yang menerima pertanyaan dari tamu (pengguna website), lalu meneruskan ke bagian yang tepat (n8n), dan ketika jawaban datang, memberikan kembali ke tamu.

**Yang dilakukan:**

```
python
# Ketika pengguna buka website → tampilkan halaman chat
@app.route("/")
def index():
    return render_template("index.html")  # ← Tampilkan halaman

# Ketika pengguna kirim pertanyaan
@app.route("/api/chat", methods=["POST"])
def chat():
    message = data.get("message")        # ← Terima pesan
    res = requests.post(N8N_WEBHOOK_URL, # ← Kirim ke n8n
                        json={"message": message})
    return jsonify(res.json())           # ← Balikkan jawaban
```

> NOTE
>
> `app.py` **tidak punya kecerdasan** — dia hanya sebagai jembatan. Semua logika ada di n8n.

**Cara jalankan:**

```
powershell
python app.py
# → Berjalan di http://localhost:5000
```

### 4. 🎨 `templates/index.html` — Tampilan Website

**Analogi:** Seperti desain ruang tamu — ini yang dilihat oleh pengguna.

**Yang ada di dalamnya:**

- **Header** → Logo UPJ + status koneksi
- **Sidebar** → Daftar topik populer yang bisa diklik
- **Hero** → Halaman sambutan dengan 4 tombol starter
- **Chat area** → Ruang percakapan
- **Input box** → Tempat ketik pertanyaan

**Cara kerjanya (JavaScript):**

```
javascript
// Ketika pengguna klik "Kirim"
async function handleSend() {
    const text = input.value;              // 1. Ambil teks
    appendMessage('user', text);           // 2. Tampilkan di chat (sisi kanan)

    const res = await fetch('/api/chat',   // 3. Kirim ke Flask
        { method: 'POST', body: {message: text} });

    const data = await res.json();         // 4. Terima jawaban
    appendMessage('bot', data.response);   // 5. Tampilkan jawaban (sisi kiri)
}
```

## 5. 📋 `n8n_chatbot_workflow.json` — Resep Alur Kerja n8n

**Analogi:** Seperti SOP (Standard Operating Procedure) yang diberikan ke n8n — "jika ada pertanyaan masuk, lakukan langkah 1, 2, 3, 4, 5 ini."

Workflow ini punya **6 node (langkah)**:

### Node 1: 🎧 Webhook Chat

```
Fungsi: Mendengarkan pesan yang datang dari Flask
Input:  POST request dari http://localhost:5000
Output: { "message": "Berapa biaya kuliah Informatika?" }
```

Seperti pintu masuk — setiap ada pertanyaan dari website, node ini yang pertama menerima.

### Node 2: 🧠 Build Cypher Query

```
Fungsi: Deteksi topik pertanyaan → buat perintah pencarian database
```

Node ini membaca kata-kata kunci dalam pertanyaan dan memutuskan apa yang harus dicari di Neo4j:

| Kata kunci terdeteksi | Query yang dibuat |
|---|---|
| "biaya", "ukt", "spp" | Cari data BiayaKuliah |
| "beasiswa", "gratis" | Cari data Beasiswa |
| "dosen", "pengajar" | Cari data Lecturer |
| "mata kuliah", "matkul" | Cari data Course |
| "syarat", "buta warna" | Cari data SyaratKhusus |
| "peminatan", "konsentrasi" | Cari data Peminatan |
| Pertanyaan umum | Cari overview University |

**Cypher** adalah bahasa query untuk Neo4j, seperti SQL tapi untuk database graf. Contoh:

```
cypher
MATCH (p:ProgramStudi)-[:HAS_TUITION_FEE]->(b:BiayaKuliah)
RETURN p.name, b.spp, b.total_semester_1
```

Artinya: "*Temukan semua Program Studi yang punya data Biaya Kuliah, tampilkan nama prodi, SPP, dan total semester 1.*"

### Node 3: 🗄️ Query Neo4j REST API

```
Fungsi: Kirim query ke database Neo4j, dapatkan data
URL:    http://127.0.0.1:7474/db/neo4j/query/v2
```

Node ini menghubungi Neo4j dan menjalankan query Cypher yang sudah dibuat. Neo4j mengembalikan data dalam format JSON.

Contoh hasil yang didapat:

```
json
[
  {
    "prodi": "Informatika",
    "spp": "3500000",
    "total_semester_1": "9450000"
  },
  {
    "prodi": "Sistem Informasi",
    "spp": "3500000",
    "total_semester_1": "9450000"
  }
]
```

### Node 4: 📝 Format Prompt

```
Fungsi: Gabungkan data Neo4j + pertanyaan asli → buat instruksi untuk AI
```

Node ini membuat "surat instruksi" untuk AI, kira-kira seperti ini:

```
Kamu adalah asisten UPJ.

DATA dari database:
[{ "prodi": "Informatika", "spp": "3500000", ... }]

Pertanyaan pengguna:
"Berapa biaya kuliah Informatika?"

Instruksi:
- Jawab dengan ramah dalam Bahasa Indonesia
- Tampilkan angka biaya dengan jelas
- Gunakan bullet point jika perlu
```

### Node 5: 🤖 SumoPod LLM

```
Fungsi: Kirim instruksi ke AI, dapatkan jawaban natural
URL:    https://api.sumopod.com/v1/chat/completions
```

AI membaca data + pertanyaan dan menghasilkan jawaban yang natural dan mudah dipahami, bukan sekedar data mentah.

**Input (data mentah):** `"spp": "3500000", "total_semester_1": "9450000"`
**Output (jawaban AI):** *"Untuk prodi Informatika, biaya SPP per semester adalah **Rp 3.500.000**. Total biaya semester pertama (termasuk uang pangkal) adalah **Rp 9.450.000**..."*

#### Node 6: 📤 Kirim ke Web

```
Fungsi: Kirim jawaban AI kembali ke Flask → lalu ke browser pengguna
Output: { "response": "Jawaban AI di sini..." }
```

## 🗂️ Neo4j — Database Graf

**Analogi:** Database biasa (seperti Excel) menyimpan data dalam tabel. Neo4j menyimpan data sebagai **jaringan hubungan**, seperti peta pikiran.

**Struktur data di Neo4j:**

```
(University: UPJ)
    │
    ├──[HAS_FACULTY]──▶ (Faculty: FTD)
    │                        │
    │                        ├──[OFFERS_PROGRAM]──▶ (ProgramStudi: Informatika)
    │                        │                            │
    │                        │                      ├──[HAS_TUITION_FEE]──▶ (BiayaKuliah)
    │                        │                      ├──[OFFERS_COURSE]────▶ (Course:
Algoritma)
    │                        │                      ├──[HAS_SPECIALIZATION]▶ (Peminatan: AI)
    │                        │                      └──[HAS_REQUIREMENT]──▶ (SyaratKhusus)
    │                        │
    │                  (Lecturer: Dr. Andi)
    │                        └──[TEACHES_IN]──▶ (ProgramStudi: Informatika)
    │
    ├──[OFFERS_SCHOLARSHIP]──▶ (Beasiswa: Jaya 100%)
    └──[HAS_FAQ]──────────────▶ (FAQ: "Bagaimana jika lolos PTN?")
```

Keunggulan Neo4j: **sangat cepat** untuk pertanyaan yang melibatkan banyak hubungan, seperti "Siapa dosen yang mengajar di prodi dengan akreditasi Unggul yang menawarkan peminatan AI?"

## 🛠️ Port yang Digunakan

| Komponen | Port | Akses |
|---|---|---|
| Website (Flask) | 5000 | localhost:5000 |
| n8n Dashboard | 5678 | localhost:5678 |
| Neo4j Browser | 7474 | localhost:7474 |
| Neo4j Bolt (internal) | 7687 | Digunakan oleh script Python |

## ⚡ Cara Menjalankan Semua (Urutan Benar)

```
1. Buka Neo4j Desktop → Start database "chatbot"  (tunggu hijau ✅)

2. Buka terminal baru:
   python build_neo4j_graph.py --password password123
   (hanya perlu sekali, untuk isi data ke Neo4j)

3. Buka terminal baru:
   python app.py
   (server web, harus tetap berjalan)

4. Buka terminal baru:
   node "C:\Users\ACER-\AppData\Roaming\npm\node_modules\n8n\bin\n8n" start
   (n8n, harus tetap berjalan)

5. Buka browser → http://localhost:5678
   → Import workflow → Set API key SumoPod → Active: ON

6. Buka browser → http://localhost:5000
   → Chatbot siap digunakan! 🎉
```

## ❓ FAQ Teknis

**Q: Kenapa pakai Neo4j bukan database biasa?**

> Database biasa bagus untuk tabel, tapi susah untuk pertanyaan "hubungan antar data". Neo4j dirancang untuk itu — cocok untuk knowledge graph kampus yang datanya saling terhubung.

**Q: Kenapa pakai n8n bukan langsung di app.py?**

> Agar logika AI bisa diubah tanpa perlu coding. Cukup ubah workflow di n8n UI, tidak perlu edit kode Python dan restart server.

**Q: Kenapa pakai SumoPod bukan ChatGPT langsung?**

> SumoPod kompatibel dengan API OpenAI, sehingga bisa diganti kapan saja. Lebih fleksibel dan bisa diatur kredensialnya di n8n.

**Q: Apakah data chatbot bisa diupdate?**

> Ya! Jalankan lagi `python scrape_pmb_upj.py` untuk ambil data baru, lalu `python build_neo4j_graph.py` untuk update database Neo4j. Workflow n8n tidak perlu diubah.